

Project Report: Ayush-Guard

Clinical Decision Support System (CDSS)

CEP Project

Date: April 18, 2026

Institution: Veermata Jijabai Technological Institute, Mumbai

1. Problem Statement

Ayush-Guard is a Clinical Decision Support System (CDSS) aimed at reconciling cross-disciplinary medication regimes (e.g., Allopathic and Ayurvedic medicines). The core problem it solves is adverse Drug-Drug Interactions (DDIs) and genetic sensitivity conflicts when a pharmacist attempts to dispense a new medicine without a holistic view of the patient's existing health profile. By integrating with the conceptual ABDM (Ayushman Bharat Digital Mission) framework, the platform extracts a patient's historical medical records and evaluates safety against accumulated health profiles.

2. Tools and Technologies

Component	Technology Used
Frontend	React, Vite, CSS
Backend Architecture	Python, FastAPI, APIRouter
Database	PostgreSQL (Supabase)
Data Validation	Pydantic (Strict format enforcement)
Testing	Pytest (Comprehensive route testing)
Deployment	Vercel (Frontend), Render / Supabase (Backend & DB)

3. Database Schema

We use Supabase (PostgreSQL) as our primary data layer. The system utilizes structured relational tables alongside flexible JSONB columns to mock the external FHIR-like records.

- **patients:** Stores structural identity data (name, abha_id). No medical history is stored directly here to maintain separation of concerns.
- **abdm_mock_records:** A JSONB representation of the external ABDM gateway. It holds arrays of medication_history, pre_existing_conditions, allergies, and raw_fhir_bundles.
- **access_requests:** Connects patients to pharmacists, representing the digital consent framework (PENDING, ACCEPTED, REJECTED states).
- **family_relationships:** Maps recursive patient IDs (e.g., Father to Son) to actively propagate genetic sensitivity warnings (e.g., G6PD Deficiency inheritance).
- **pharmacists & admins:** Stores credentialing and license information for registered healthcare vendors.

4. Solution Idea & Architecture

The project strongly separates concerns into a static UI layer (React/Vite) for highly responsive SPA mechanics, and a heavy computational API layer (FastAPI) for deterministic safety scoring.

1. Data ETL Pipeline: final_generate.py ingests raw datasets (DrugBank, PharmGKB) into optimized, memory-resident JSON dictionaries (brand_to_salt, ddi_map, clinical_map).

2. Step 1: Pharmacist issues an Access Request to a specific patient's ABHA ID.
3. Step 2: Patient grants consent, unlocking their ABDM Mock Records.
4. Step 3: Pharmacist inputs a brand name into the DashboardPanel.
5. Step 4: Frontend calls POST /api/check-drug.
6. Step 5: Backend resolves brand to salt and cross-references patient's active meds and genomics against the memory-resident JSON maps.
7. Step 6: Backend checks family_relationships for inherited risks.
8. Step 7: A structured alert (severity_tier, risk_probability, message) is returned to the user interface.

5. Challenges Faced

- **Lack of Sensitive Clinical Data:** Due to privacy constraints, real patient datasets were unattainable. This occasionally caused the ML/NLP layers to hallucinate edge cases, forcing reliance on structured, mock demo injections to maintain deterministic safety scoring.
- **CORS Debugging:** Managing strict Cross-Origin Resource Sharing (CORS) rules between the isolated Vercel frontend and the Render-hosted backend during integration.
- **Data Parsing & Formats:** Handling semi-structured clinical JSON dumps required heavy usage of Pydantic to strictly enforce data validation across API boundaries.
- **Performance at Scale:** Fetching bulk user records strained network resources, requiring the implementation of backend pagination for efficient data retrieval in the GET /api/patients route.
- **Asynchronous Testing:** Ensuring that complex chained API endpoints evaluated safely under load necessitated rigorous Pytest implementations.

6. Future Improvements

- **Real ABDM Integration:** Swapping mocked JSON records for active HTTPS requests to the National Health Authority (NHA) gateway, decrypting FHIR JSONs via Diffie-Hellman keys.
- **Deep Learning Embeddings:** Upgrading text-matching to use a medical BERT model (e.g., ClinicalBERT) to identify semantic similarities within unstructured physician notes.
- **Advanced State Management:** Utilizing React Query or Redux to cache patient histories locally, drastically reducing redundant API trips.
- **Dosage-Specific Risk Calculation:** Upgrading the ML engine to require drug strength arrays, distinguishing between safe low doses and toxic thresholds.

7. Conclusion

Ayush-Guard successfully demonstrates the critical need for an interoperable Clinical Decision Support System. By merging modern web architecture (FastAPI/React) with robust deterministic mapping and mock ABDM integration, the prototype offers a scalable foundation for preventing cross-disciplinary drug interactions and protecting patient safety.

Project Deliverables & Resources

[Drive Link: Demo Video] [https://drive.google.com/drive/folders/1tY_klodFTW-LLNPZm9NSv7Vj5MuLFM_M?usp=drive_link]

[Drive Link: ER Diagram]
[https://drive.google.com/file/d/1XNdwmZSAPkVtINx7gCbRf1AlAfsESluM/view?usp=drive_link]

[Drive Link: SQL schema] [https://drive.google.com/file/d/1tyNdVT94ENgaKJ4r-4viz2iJ_hFTIzKn/view?usp=drive_link]

[Drive Link: Workflow Diagram] [https://drive.google.com/file/d/1LqGGX80y2SGL73-mYT8fb15WD7QPK8gZ/view?usp=drive_link]

Hosted Site: <https://cep-batch-c-sy-it-2028.vercel.app/>

Github Repo: https://github.com/Once-1296/CEP_Batch_C_SY_IT_2028

Scripts

ER Diagram (Mermaid Script)

```
erDiagram
    pharmacists ||--o{ patients : "registers"
    pharmacists ||--o{ access_requests : "makes"
    patients ||--o{ access_requests : "receives"
    patients ||--o{ abdm_mock_records : "has"
    patients ||--o{ family_relationships : "has relative"

    patients {
        uuid id PK
        text abha_id UK
        text name
        text phone
        uuid registered_by FK
    }
    abdm_mock_records {
        text abha_id PK, FK
        jsonb basic_health_details
        jsonb pre_existing_conditions
        jsonb medication_history
        jsonb allergies
    }
    access_requests {
        uuid id PK
        uuid pharmacist_id FK
        uuid patient_id FK
        text status
    }
    family_relationships {
        uuid id PK
        uuid patient_id FK
        uuid relative_id FK
        text relationship_type
    }
    pharmacists {
        uuid id PK
        text name
        text license_number UK
    }
    admins {
        uuid id PK
        text username UK
    }
```

Workflow Diagram (Mermaid Script)

```
flowchart TD
    Start([Start([Pharmacist wants to dispense drug])]) --> Step1[Step1[Request Access to Patient Profile]]
    Step1 --> Step2{Step2{Consent Granted?}}
    Step2 -- Yes --> Step3[Step3[Fetch ABDM Records JSON]]
    Step2 -- No --> End1([End1([Access Denied])])
```

```
Step3 --> Step4[Input Brand: POST /api/check-drug]
Step4 --> Step5[Resolve Brand to Salt via final_generate maps]
Step5 --> Step6[Cross-reference DDI, Genotype & Allergy Maps]
Step6 --> Step7[Check family_relationships for Inheritance Risks]
Step7 --> Step8[Calculate Safety Score & Output Alerts]
Step8 --> End([Dispense or Deny Medication])
```

```
classDef step fill:#f9f9f9,stroke:#333,stroke-width:2px;
class Step1,Step2,Step3,Step4,Step5,Step6,Step7,Step8 step;
```